ISSUE 日本語解説資料

Issue #8 マルチクラウド設計制約を定義する

GitHub で closed になっている issue 記録を、そのまま参照用 PDF に変換したもの。

Issue #8　更新: 2026/03/08　ソース: docs/portal/issues/issue-08-multicloud-design-constraints.md

GitHub: https://github.com/PLAYER1-r7/MultiCloudProject/issues/8

計画　portal　architecture

この ISSUE の要点

今回は AWS で開始するが、将来 Azure と GCP に広げる前提があるため、今の設計で固定化しすぎない必要がある。

目標

将来のマルチクラウド展開を妨げない制約を今の段階で明文化する。

完了状態

[docs/portal/10_MULTICLOUD_CONSTR
(docs/portal/10_MULTICLOUD_CONSTR
を multicloud constraint 判断の中心文書として扱う

1. この Issue が扱っていたこと

まず Summary と Goal から、この Issue が何を決めるためのものだったかを短く押さえる。

今回は AWS で開始するが、将来 Azure と GCP に広げる前提があるため、今の設計で固定化しすぎない必要がある。

目標

将来のマルチクラウド展開を妨げない制約を今の段階で明文化する。

2. 完了判断の見方

Tasks、Definition of Done、Current Status の記述から、どこまで完了済みとして扱っているかを先に読む。

TASKS

6/6 項目が完了済みとして記録されている。

DEFINITION OF DONE

6/6 項目が完了済みとして記録されている。

> docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md を multicloud constraint 判断の中心文書として扱う / local issue file には task contract、discussion draft、resolution、evidence mapping、final review result を追加済み / 初回リリースの multicloud judgment は `AWS-specific deployment is allowed` と `product/app-layer constraints remain cloud-neutral` で確定した / naming、URL、auth、config、observability、frontend boundary の各論点は後続 issue へ引き継げる粒度で文書化した / provider-specific output は frontend contract に直接露出させず、delivery または infra 側で neutral contract に変換する前提を明記した / final checkbox review は完了し、Tasks と Definition of Done は満了と判断した / GitHub Issue 8 は最終判断に基づいて close し、Issue 8 は完了と判断する

3. 依存関係と読み進め方

Dependencies や前提 Issue がある場合は、ここで関連先を先に把握しておくと読みやすい。

依存関係

- Issue 4
- Issue 6
- Issue 7

読み方の注意

この解説書は原文の意思決定や完了状態を読みやすくするための補助資料である。最終的な正本は原文 Issue にあり、判断変更時は原文側を先に更新する。

4. 用語集

原文に残る英語や実務用語のうち、訳し切らずに補助説明を付けた方が読みやすい語をここにまとめる。

原語	日本語の扱い	補足
staging	ステージング	本番前に動作や反映を確認するための検証環境。
production	本番環境	実際の利用者に公開される運用環境。
workflow	ワークフロー	GitHub Actions などで定義された自動処理の流れ。
owner	責任者	意思決定または更新責任を持つ担当者。文脈により product owner などを含む。
review	レビュー	内容や実装が要件を満たすか確認する工程。
evidence	証跡	判断や完了状態の根拠になるログ、表、文書、出力結果。
mapping	対応付け	チェック項目とその根拠を結び付けて整理すること。
draft	下書き	まだ確定前だが議論やレビューの基礎として置かれた文書。
release	リリース	利用者に公開するまとまりとしての提供単位。
close approval	クローズ承認	Issue を完了扱いとして閉じてよいという明示承認。
current status	現在の状態	Issue 文書時点での完了度合いや同期状況を示す節。
definition of done	完了条件	その Issue を完了とみなすために満たすべき条件。
task contract	タスク契約	対象、目的、スコープ、検証計画、リスクを明文化した作業契約。

5. 原文

以下に Issue 原文をそのまま掲載する。詳細な根拠、証跡、チェックリストはこの原文を参照する。

概要

今回は AWS で開始するが、将来 Azure と GCP に広げる前提があるため、今の設計で固定化しすぎない必要がある。

目標

将来のマルチクラウド展開を妨げない制約を今の段階で明文化する。

タスク契約

```
タスク契約

メタデータ
- タスク ID: ISSUE-08
- タイトル: マルチクラウド設計制約の初期整理
- 依頼者: repository owner
- 担当者: AI agent
- 対象アプリ: portal-web
- 対象環境: planning
- 優先度: 高

目的
- 解決する問題: AWS-first で進める初回リリースにおいて、どこまでを AWS 固有実装として許容し、どこからを cloud-neutral constraint として固定するかを明確にする
- 期待する価値: Issue 9 以降の IaC、CI/CD、監視、セキュリティ方針が AWS 先行の都合で app model まで固定化しないようにする

スコープ
- 含むもの: 命名、URL、認証、設定値管理、監視観点、frontend boundary の portability constraints 整理
- 含まないもの: Azure/GCP 実装、runtime multi-cloud 配備、各クラウドの詳細比較、実装着手
- 編集可能パス: docs/portal/issues/issue-08-multicloud-design-constraints.md, docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md
- 制限パス: apps/, infra/, deployment workflows

受け入れ条件
- [x] 条件 1: AWS 固有でよいレイヤと cloud-neutral に保つレイヤを分けて議論開始できる状態になっている

実装計画
- 変更見込みファイル: docs/portal/issues/issue-08-multicloud-design-constraints.md, docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md
- アプローチ: Issue 7 の static-first 結論を前提に、product/app/infrastructure/operations の4層で portability boundary を切り分ける
- 採用しなかった代替案と理由: 今の段階で Azure/GCP 実装差分まで先に設計する案は、AWS-first planning の焦点をぼかしやすいため採らない

検証計画
- 実行するテスト: git status --short; GH_PAGER=cat gh issue view 8 --repo PLAYER1-r7/MultiCloudProject --json number,state,title,updatedAt,url
- 確認するログ/メトリクス: TBD
- 失敗時の切り分け経路: Issue 3, Issue 4, Issue 7 の既定方針へ戻り、constraint が product requirement ではなく implementation preference を混ぜていないかを確認する

リスクとロールバック
- 主なリスク: portability の名目で抽象化しすぎ、AWS-first の実装判断まで不必要に遅くすること
- 影響範囲: Issue 9 以降の設計粒度、命名、設定値、監視、frontend implementation boundary
- 緩和策: first-release に必要な constraint だけを残し、runtime multi-cloud 実装や provider abstraction は later phase として切り分ける
- ロールバック手順: 抽象化が過剰だと判明した場合は、user-facing model と frontend boundary だけを残し、infra-level portability detail は後続 issue へ戻す
```

作業項目

- [x] 命名規則の制約を定義する
- [x] URL 設計の制約を定義する
- [x] 認証設計の制約を定義する
- [x] 設定値管理の制約を定義する
- [x] 監視項目の共通化方針を定義する
- [x] フロントエンドからクラウド固有実装を直接参照しない方針を定義する

完了条件

- [x] AWS 固定化しない設計方針がアプリ層とインフラ層を分けて説明されている
- [x] 命名規則と URL 設計の制約が利用者視点と実装視点の両方で整理されている
- [x] 認証と設定値管理の制約が特定クラウドに依存しない形で説明されている
- [x] 監視やヘルスチェックの共通化方針が整理されている
- [x] フロントエンドからクラウド固有実装を直接参照しない原則が説明されている
- [x] 将来 Azure と GCP へ拡張する際の前提条件として参照できる

Issue 8 Discussion Draft

このセクションは、Issue 8 の議論を始めるためのたたき台である。Issue 7 の結論により、初回リリースは static-first、no custom API、no application persistence を前提とする。したがって Issue 8 で決めるべきことは「multi-cloud を今すぐ実装する方法」ではなく、「AWS-first で進めても後から Azure/GCP へ広げられるように、どの境界を固定しておくか」である。

1. 現時点の前提整理

- 初回リリースは AWS-first であり、runtime multi-cloud deployment はスコープ外である
- ただし product model と app-layer naming は AWS サービス名に寄せすぎない必要がある
- frontend は static-first TypeScript 配信であり、cloud-vendor SDK や secret を抱えない前提である
- backend と application data persistence は未採用であるため、Issue 8 では API portability よりも route、config、observability、boundary の portability が中心になる

2. まず固定したい portability boundary

提案:

- infrastructure delivery choice は AWS 固有でよい
- product structure と user-facing route は cloud-neutral に保つ
- frontend configuration contract は provider-neutral に保つ
- monitoring と health check の naming は app outcome 中心に保つ

この切り分けを採る理由:

- 初回リリースの実装速度は AWS-specific infrastructure を許容した方が高い
- 一方で user-facing model まで AWS に寄せると、あとで Azure/GCP へ広げる時に仕様変更が混ざる
- 「deployment target の差」と「product/app model の差」を分離しておけば、後続 issue の責務も明確になる

3. 命名規則のたたき台

提案:

- user-facing page 名、navigation 名、document 名に AWS サービス名を埋め込まない
- app module、environment、artifact 名は可能な限り neutral naming を使う
- provider 名が必要な場合は infra 層や workflow 層に閉じ込める

具体例:

- 採る: portal, staging, production, static-site, delivery, public-assets
- 避ける: cloudfront-page, s3-portal-ui, aws-auth-route, dynamodb-content

論点:

- repository 内の infra module や workflow file は provider 名を含んでもよいか
- どの層まで neutral naming を強制するか

現時点の第一案:

- user-facing と app-facing naming は neutral
- infra-facing naming は provider 名を含んでもよいが、frontend や docs の product language へ漏らさない

4. URL 設計のたたき台

提案:

- URL は product area と user intent を表し、cloud provider や deploy topology を反映しない
- 初回リリースの route は shallow かつ public-first を維持する
- 後から auth や dynamic feature が増えても URL 全体を作り直さなくて済むようにする

具体例:

- 採る: /, /overview, /guidance
- 避ける: /aws/overview, /cloudfront/help, /s3-assets, /public-bucket-guide

現時点の第一案:

- route は利用者の目的に基づいて定義し、provider detail は URL に出さない

5. 認証設計制約のたたき台

提案:

- 初回リリースは no end-user login を維持する
- 将来 auth を導入する場合でも、protected area の概念を provider-neutral に保つ
- frontend route 設計時に Cognito 前提の文言や情報設計を埋め込まない

理由:

- Issue 3 の時点では auth 自体が deferred である
- ここで Cognito 前提を app model に埋め込むと、将来 Azure AD B2C や GCP 系 identity と差し替えにくくなる

現時点の第一案:

- 認証は導入時点まで抽象概念に留め、ログイン導線、保護領域、role naming を provider 固有名にしない

6. 設定値管理制約のたたき台

提案:

- frontend から参照する設定値は environment-driven かつ public-only に限定する
- secret は repository と frontend bundle に置かない
- domain、base path、asset origin、将来の API base URL などは provider 名ではなく役割名で扱う

具体例:

- 採る: PUBLIC_BASE_URL, PUBLIC_ASSET_BASE_URL, PUBLIC_CONTACT_URL
- 避ける: AWS_CLOUDFRONT_URL, S3_WEBSITE_ENDPOINT, AWS_API_GATEWAY_URL

補足:

- infra 側で provider-specific output が存在しても、frontend へ渡す時は neutral contract に変換する方が望ましい

7. 監視項目共通化のたたき台

提案:

- health check は provider resource の死活よりも user-facing outcome を主語にする
- monitoring label と dashboard は app name、environment、route、availability を中心に定義する
- alert は CloudFront や Azure Front Door などの provider 差分を吸収できる粒度に寄せる

具体例:

- 採る観点: home page reachable, overview page reachable, guidance page reachable, static asset load success
- 依存しすぎない観点: specific AWS metric name を中心にした監視説明

現時点の第一案:

- provider metrics は backend implementation detail として使ってよいが、monitoring policy の主文脈は user-facing availability に寄せる

8. Frontend boundary のたたき台

提案:

- frontend code は cloud-vendor SDK へ直接依存しない
- frontend は deployment target を意識しすぎず、public static app として振る舞う
- provider 固有処理が必要になっても、まず infra or delivery layer で吸収できないかを優先する

理由:

- 初回リリースの frontend は static-first であり、cloud SDK を抱える合理性が薄い
- cloud SDK 依存を app code に入れると、後続の provider 差し替えが UI architecture の問題に化ける

現時点の第一案:

- frontend は cloud-neutral

- infra と workflow は provider-specific 実装を持ってよい
- app と infra の接点は config contract に限定する

9. 今回は決めないこと

- Azure と GCP の具体的な resource mapping
- provider ごとの IaC module 分割詳細
- multi-cloud 同時運用時の failover 戦略
- auth 導入時の identity provider 選定
- vendor-neutral abstraction layer の実装方式

この切り分けを置く理由:

- 今の論点は portability boundary であり、implementation parity ではないため
- 詳細に入りすぎると Issue 9 以降の責務を先食いするため

10. 議論開始用の暫定結論

- AWS-specific deployment は許容する
- product structure、URL、frontend config contract、frontend architecture、monitoring wording は cloud-neutral に保つ
- provider 名は infra と workflow の内部にはあってよいが、user-facing model と frontend contract へ漏らさない
- multi-cloud portability は「runtime parity」ではなく「app model を壊さずに後から置き換え可能であること」を目標にする

11. この場で確認したい論点

- neutral naming をどの層まで強制するか
- provider-specific output を frontend contract に変換する運用を必須とみなすか
- monitoring policy を user journey 中心に固定してよいか
- auth を later phase に置いたままでも route naming の constraint は十分か
- 「AWS-specific infrastructure は許容、app model は neutral」という境界で Issue 9 以降を進めて問題ないか

Resolution

Issue 8 の判断結果は次の通りとする。

- 初回リリースの deployment target は AWS 固有実装でよい
- ただし product structure、user-facing route、frontend configuration contract、monitoring wording、frontend architecture は cloud-neutral に保つ
- provider 名や provider 固有 resource 名は infra 層、workflow 層、運用実装の内部に閉じ込め、user-facing model と app-facing contract には持ち込まない
- frontend が参照する設定値は role-based かつ provider-neutral な名前で定義し、provider-specific output は delivery または infra 側で変換して受け渡す
- 初回リリースの frontend code は cloud-vendor SDK に直接依存しない
- 監視、health check、alert の主語は provider resource ではなく user-facing availability と route outcome に置く
- 将来 auth を導入する場合でも、protected area、role、login 導線などの app model は provider 固有語彙で固定しない
- multi-cloud portability の目標は runtime parity ではなく、product/app model を壊さずに Azure または GCP へ後から拡張可能にしておくこととする

この判断で明確になること:

- AWS-first の実装速度は維持できる
- app model を AWS 用語で汚染しないため、後続 issue で Azure/GCP 拡張時の仕様変更を最小化できる
- Issue 9 以降は provider abstraction の作り込みではなく、boundary と contract の維持を重視して進められる

Issue 9 以降への引き継ぎ事項:

- naming review では user-facing と app-facing の cloud-neutral 性を優先確認する
- config contract review では provider-specific variable 名を frontend へ露出させない
- monitoring review では provider metric の採用可否より先に user journey と availability outcome を定義する
- auth や dynamic feature が later phase で追加されても、route と app model を provider 固有語彙へ寄せない
- Azure/GCP 対応の具体的 resource mapping や failover 設計は後続 issue で扱い、この Issue では portability boundary の合意に留める

証跡対応表

The tables below identify the evidence used for final checkbox review and should remain aligned with the checked state above.

作業項目対応表

Checklist item	Primary evidence section	Why this is the evidence	Review state
命名規則の制約を定義する	3. 命名規則のたたき台, Resolution, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections define cloud-neutral naming boundaries, show acceptable and rejected examples, and record the final decision to keep provider names out of user-facing and app-facing identifiers.	Accepted for final review
URL 設計の制約を定義する	4. URL 設計のたたき台, Resolution, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections state that routes should describe product areas and user intent rather than provider topology, and they capture that rule in the final decision.	Accepted for final review
認証設計の制約を定義する	5. 認証設計制約のたたき台 and Resolution in docs/portal/issues/issue-08-multicloud-design-constraints.md	These sections explain that authentication remains deferred for the first release and that future protected-area concepts must stay provider-neutral.	Accepted for final review
設定値管理の制約を定義する	6. 設定値管理制約のたたき台, Resolution, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections define role-based, provider-neutral configuration naming and require provider-specific outputs to be translated before they reach the frontend contract.	Accepted for final review
監視項目の共通化方針を定義する	7. 監視項目共通化のたたき台, Resolution, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections define observability around user-facing outcomes and availability instead of provider-specific metric terminology.	Accepted for final review
フロントエンドからクラウド固有実装を直接参照しない方針を定義する	8. Frontend boundary のたたき台, Resolution, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections state that frontend code should remain cloud-neutral, avoid direct cloud SDK dependency, and connect to infrastructure only through a neutral config contract.	Accepted for final review

完了条件対応表

Checklist item	Primary evidence section	Why this is the evidence	Review state
AWS 固定化しない設計方針がアプリ層とインフラ層を分けて説明されている	2. まず固定したい portability boundary, Resolution, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections explicitly separate AWS-specific infrastructure choices from cloud-neutral product and app-layer constraints.	Accepted for final review
命名規則と URL 設計の制約が利用者視点と実装視点の両方で整理されている	3. 命名規則のたたき台, 4. URL 設計のたたき台, and Resolution in docs/portal/issues/issue-08-multicloud-design-constraints.md	These sections define both user-facing naming and routing rules and the implementation-side restriction that provider naming stays inside infra and workflow layers.	Accepted for final review
認証と設定値管理の制約が特定クラウドに依存しない形で説明されている	5. 認証設計制約のたたき台, 6. 設定値管理制約のたたき台, and Resolution in docs/portal/issues/issue-08-multicloud-design-constraints.md	These sections define provider-neutral auth concepts and provider-neutral configuration contracts without embedding AWS-specific identity or variable names into the app model.	Accepted for final review

Checklist item	Primary evidence section	Why this is the evidence	Review state
監視やヘルスチェックの共通化方針が整理されている	`7. 監視項目共通化のたたき台`, `Resolution`, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections explain that health and alert design should be based on user journey and app availability, with provider metrics treated as implementation detail.	Accepted for final review
フロントエンドからクラウド固有実装を直接参照しない原則が説明されている	`8. Frontend boundary のたたき台`, `Resolution`, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections make the frontend cloud-neutral rule explicit and prohibit direct dependence on cloud-vendor SDKs for the first release.	Accepted for final review
将来 `Azure` と `GCP` へ拡張する際の前提条件として参照できる	`Resolution`, `Issue 9 以降への引き継ぎ事項`, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md	These sections define the portability boundary and the downstream review rules needed to preserve it when Azure or GCP work begins later.	Accepted for final review

Final Review Rule For Issue 8

- The presence of mapped evidence is not by itself enough to check the box.
- Final checkbox review should confirm that the mapped section still reflects the latest agreed wording.
- If a mapped section changes materially, the table should be rechecked before marking the checklist item complete.
- If the local issue definition changes after the last remote sync, the GitHub issue body should be synced again before close or any equivalent final-state transition.

最終レビュー結果

Final checkbox review completed against the latest wording in docs/portal/issues/issue-08-multicloud-design-constraints.md and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md. The table below records the document-level validation judgment. Human close approval remains a separate action.

Checklist area	Final judgment	Evidence basis
Portability boundary decision	Satisfied	`2. まず固定したい portability boundary`, `Resolution`, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md confirm that AWS-specific deployment is allowed while product and app-layer contracts remain cloud-neutral.
Naming decision	Satisfied	`3. 命名規則のたたき台` and `Resolution` confirm that provider names stay out of user-facing and app-facing naming while remaining permissible inside infra and workflow layers.
URL decision	Satisfied	`4. URL 設計のたたき台` and `Resolution` confirm that route design follows product intent rather than provider topology.
Auth and config decision	Satisfied	`5. 認証設計制約のたたき台`, `6. 設定値管理制約のたたき台`, and `Resolution` confirm provider-neutral auth concepts and provider-neutral frontend config naming.
Observability decision	Satisfied	`7. 監視項目共通化のたたき台`, `Resolution`, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md confirm that monitoring is framed around user-facing availability and route outcomes.

Checklist area	Final judgment	Evidence basis
Frontend boundary decision	Satisfied	`8. Frontend boundary` のたたき台, `Resolution`, and docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md confirm that frontend code remains cloud-neutral and does not directly depend on cloud-vendor SDKs.
Future expansion readiness	Satisfied	`Resolution` and `Issue 9` 以降への引き継ぎ事項 confirm that Azure/GCP expansion should preserve the agreed portability boundary instead of redesigning the app model.

プロセスレビュー記録

- The Task Contract and discussion structure now exist in this file and are sufficient for auditability of the final planning judgment.
- This issue was hardened from discussion draft to final review format during the planning flow, so the current structure should be read as a consolidated planning record.
- The open questions listed in `11. この場で確認したい論点` were not answered one by one in a separate discussion log before the resolution was finalized; instead, the resolution was fixed by direct decision flow after the user requested to proceed from resolution first.
- The resulting resolution still covers those open questions substantively, but the record should be read as a direct-decision planning close rather than a fully expanded question-by-question discussion transcript.
- Final review output is separated from evidence mapping so later reviewers can distinguish completion judgment from source locations.

現在の状態

- docs/portal/10_MULTICLOUD_CONSTRAINTS_DRAFT.md を multicloud constraint 判断の中心文書として扱う
- local issue file には task contract、discussion draft、resolution、evidence mapping、final review result を追加済み
- 初回リリースの multicloud judgment は `AWS-specific deployment is allowed` と `product/app-layer constraints remain cloud-neutral` で確定した
- naming、URL、auth、config、observability、frontend boundary の各論点は後続 issue へ引き継げる粒度で文書化した
- provider-specific output は frontend contract に直接露出させず、delivery または infra 側で neutral contract に変換する前提を明記した
- final checkbox review は完了し、Tasks と Definition of Done は満了と判断した
- GitHub Issue 8 は最終判断に基づいて close し、Issue 8 は完了と判断する

依存関係

- Issue 4
- Issue 6
- Issue 7